

Vision Documentation	2
Interactive House	2
Revision History	2
Product Overview — Server & Database Subsystem	2
1. System Concept	2
1.1 Stakeholders	2
1.2 Introduction	2
2. System Purpose and Responsibilities.....	3
2.1 API-based Coordination	3
3. Technical Overview	3
3.1 General Characteristics.....	3
3.2 Normal Operation Mode	3
3.3 Fault and Recovery Handling.....	4
4. Basic Requirements	4
5. Implemented Features	4
6. Future Work	4

Vision Documentation

Interactive House

GitHub: [Interactive House](#) — see `SG1_Server_DB/` for server & database code

Revision History

Date	Version	Description	Author
08/02/2026	1.0	Laying down foundation of the vision	Katerina Arvay, Dario Ostojic, Andre Sandblom
17/03/2026	1.1	Updated vision to reflect current hybrid architecture and planned gateway-centered evolution	Katerina Arvay, Dario Ostojic, Andre Sandblom
23/04/2026	1.2	Updated vision to reflect the use of a backup database	Katerina Arvay, Dario Ostojic, Andre Sandblom
08/05/2026	1.3	Final revision: corrected architecture from codebase; backup DB noted as not implemented; two-collection Firestore structure; LCD device and Cloudflare Tunnel confirmed; Implemented Features and Future Work sections added	Katerina Arvay, Dario Ostojic, Andre Sandblom

Product Overview — Server & Database Subsystem

1. System Concept

1.1 Stakeholders

The primary stakeholders of the Server and Database subsystem are the development teams responsible for other Smart House components, including device controllers, user interfaces, and automation services. Additional stakeholders include system administrators and course instructors who evaluate the design and implementation. End users, such as homeowners, are indirect stakeholders who rely on this subsystem for stable and reliable system behavior.

1.2 Introduction

The Server and Database subsystem represents the core backend of the Smart House system. It functions as a centralized platform that enables communication, coordination, and data persistence for distributed smart house devices. The subsystem is designed to support an interactive house environment by managing system state and acting as a mediator between devices and higher-level services.

2. System Purpose and Responsibilities

The main purpose of the Server and Database subsystem is to provide a dependable backbone for the Smart House system. Its responsibilities include coordinating interactions between components, maintaining consistent system state, and ensuring that data is persistently stored and retrievable. By centralizing these responsibilities, the subsystem reduces complexity in individual devices and improves overall system maintainability. The subsystem ensures that system data is stored reliably and can be recovered in case of failures.

2.1 API-based Coordination

The Server and Database subsystem exposes a set of well-defined service interfaces that enable communication between system components, including client applications, devices, and supporting services. These interfaces allow users and devices to authenticate, register, and interact with smart house functionality.

The server acts as the central coordination point and maintains a consistent system state across all connected components. All interactions with house devices — such as controlling lights, activating alarms, or retrieving sensor data — are managed through the backend, where commands are validated and access rules are enforced.

The subsystem is designed to support a unified and reliable flow of information between components, ensuring that system state reflects actual device behavior. Over time, the architecture aims to further strengthen centralized control and validation mechanisms to improve consistency, reliability, and maintainability.

3. Technical Overview

The Server and Database subsystem is designed as a backend service that operates continuously within the smart house environment. It is responsible for handling concurrent communication with multiple components and managing structured system data.

The web application is deployed as a cloud-hosted web service (Render), while mobile clients are distributed as installable applications. The gateway remains locally hosted to maintain direct communication with physical devices.

3.1 General Characteristics

- Centralized coordination of smart house components
- Persistent storage of device states, configurations, and system data
- Support for concurrent interactions from multiple devices and services
- Designed for extensibility as new devices are added
- Exposes a centralized API for authentication, device control and state observation

3.2 Normal Operation Mode

During normal operation, the server receives requests and events from connected components, processes them, and updates the system state accordingly. Device states are stored across two Firestore collections: `users/{username}` (updated via the SG1 API) and `devices/arduino`

(updated by the SG3 gateway after confirmed serial execution). SG2 mobile and web clients observe the shared devices/arduino document in real time. This mode represents the standard operational behaviour of the Smart House system.

3.3 Fault and Recovery Handling

The subsystem is designed to handle temporary device disconnects, server restarts, and data storage failures without loss of critical system data. Persistent storage relies on Firebase Firestore. Note: the planned secondary relational database (Supabase) has not been implemented — the supabase/ folder contains only placeholder stub code. A backup and recovery strategy remains as future work.

4. Basic Requirements

This subsystem must support reliable communication, persistent data storage, and coordinated interaction between smart house components. A detailed list of functional and supplementary requirements is specified in the corresponding requirements documents.

5. Implemented Features

The following features are confirmed implemented and operational as of v1.3:

- User registration with bcrypt hashing (10 rounds); protected by REGISTRATION_API_KEY
- User authentication via JWT tokens (2-hour expiry, HS256)
- Password update (PATCH /users/update-password) with current password verification
- Client-side logout; server relies on 2-hour token expiry
- Device state control: white_light, orange_light (PWM 0–255), door, window, fan_INA, fan_INB, buzzer
- LCD text display (ledTextDisplay) — gateway forwards to Arduino LCD1602 via M[text]] serial command
- DELETE /users/device for removing device fields
- Sensor telemetry (gas, steam, motion) read by SG3 gateway; stored to Firestore devices/arduino
- Firebase Functions deployed at us-central1-smarthouse-617fe.cloudfunctions.net
- SG2 web application deployed on Render; SG3 gateway exposed via Cloudflare Tunnel

6. Future Work

Secondary relational database (Supabase): NOT implemented. Only placeholder stub code exists.

- Deploy proper Firestore security rules to the SG1 project (currently wide open until Oct 2026)
- Implement Supabase backup and recovery database
- Server-side JWT revocation (proper logout support)
- Full scalability and load testing